ZETHETA

Internship Selection Assignment

Distributed Candidate Evaluation Platform

Issued by	Zetheta Engineering Team
Duration	3 calendar days from receipt of this document
Submission Format	GitHub repository link — repository will be transferred to Zetheta upon submission
Repository Transfer	By submitting, you agree to transfer ownership of the repository to the Zetheta GitHub organisation for evaluation purposes
Coding Tools	Permitted (see Section 10 for conditions)

1. Context & Purpose

Zetheta is building production-grade distributed infrastructure for real-time talent and evaluation systems. As part of our internship selection process, we invite candidates to design and implement a distributed platform from scratch — end to end, under real engineering constraints.

This assignment is intentionally open-ended. We are less interested in a perfect solution and more interested in how you think, how you document your decisions, and how you handle complexity under time pressure. Your process matters as much as your output.

2. Problem Statement

Design and implement a production-grade distributed candidate evaluation platform consisting of multiple independent applications operating within a shared system. The platform must support:

- Candidate assessments delivered through a secure, sessionless cross-application flow
- Real-time employer insights updated without page refresh
- An asynchronous, fault-tolerant evaluation pipeline
- Secure transitions between independent applications within a single user workflow

© 2025 Zetheta. For selection purposes only.

3. Core Architecture Requirements

3.1 Monorepo Structure (Mandatory)

The system must be implemented as a single monorepo with clear service boundaries, reusable shared modules, and a structure that supports multi-team development. The required layout is:

```
apps/
  candidate-portal/       # User login, assessment entry point
  assessment-engine/      # MCQ interface, answer submission
  employer-dashboard/     # Candidate funnel, real-time updates

services/
  api-gateway/            # Unified ingress, routing
  evaluation-worker/      # Async scoring pipeline
  auth-service/           # Token issuance, verification

packages/
  shared-types/           # Shared TypeScript types / contracts
  utils/                  # Shared utility modules
```

3.2 Application Descriptions

Candidate Portal

Provides user login and serves as the entry point to the assessment flow. When a candidate clicks Start Assessment, they must be transitioned into the Assessment Engine — a separate, independently deployed application — without re-authentication.

Assessment Engine (Separate Application)

Renders an MCQ-based assessment interface, handles answer submission, and triggers the evaluation pipeline. Logic should be simple (MCQs only); the focus is on system integration and secure cross-app flow, not question complexity.

HR Dashboard

Displays a real-time candidate funnel (Applied → Attempted → Evaluated) with a filterable candidate list and score visibility. The dashboard must update automatically without refresh when new evaluation results arrive.

© 2025 Zetheta. For selection purposes only.

4. Cross-Application Workflow (Critical Requirement)

The Candidate Portal and the Assessment Engine are independent applications. When a candidate initiates an assessment, the following constraints must all be satisfied simultaneously:

Constraint	Requirement
Server-side generation	The access mechanism must be generated server-side — never client-generated.
Short-lived	Must expire within a tight window (e.g., 60–120 seconds) to prevent misuse.
Single-use	Must be invalidated immediately upon use. Reuse must be rejected.
Integrity-guaranteed	Must cryptographically bind the token to the candidate identity and session.
Replay-attack resistant	Must include a nonce or equivalent mechanism to prevent replay attacks.

This mechanism is a core evaluation dimension. Submissions without a working, correctly reasoned cross-application transition will be penalised significantly.

5. Distributed Evaluation Pipeline

When a candidate submits an assessment, the following sequence must execute reliably:

```
1. Store submission  →  PostgreSQL
2. Publish event     →  Redis (pub/sub or queue)
3. Worker picks up   →  evaluation-worker service
4. Score computed    →  stored back to PostgreSQL
5. Notify dashboard  →  real-time update via WebSocket / SSE
```

5.1 Pipeline Requirements

- Asynchronous execution — submission response must not block on scoring
- Idempotent — duplicate events must not produce duplicate score records
- Retry-safe — transient failures must trigger a retry; permanent failures must be logged
- Fault-tolerant — Redis downtime must degrade gracefully without data loss

© 2025 Zetheta. For selection purposes only.

6. Data Layer

6.1 PostgreSQL

Design an efficient, well-indexed schema covering the following entities:

- Candidates — identity, status, timestamps
- Applications — links candidates to assessment sessions
- MCQ Questions — question text, options, correct answer
- Responses — per-question candidate answers with submission metadata
- Scores — computed result, evaluation timestamp, worker metadata

Indexed queries are mandatory. Full table scans on any hot path will be penalised.

6.2 Redis

- Event queue or pub/sub channel for evaluation pipeline events
- Real-time update delivery to the Employer Dashboard
- Optional: short-lived token store for cross-application tokens

---

7. Real-Time System & Observability

7.1 Real-Time Updates

The Employer Dashboard must update without a page refresh in response to:

- A candidate submitting an assessment
- An evaluation completing and a score being stored

Accepted approaches: WebSockets, Server-Sent Events (SSE), or Redis pub/sub bridged to the client.

7.2 Observability (Mandatory)

The system must include structured observability across all services:

- Structured logging — JSON format, consistent field names, log levels
- Error tracking — uncaught errors captured with context and stack traces
- Metrics — track API latency (p95 target: <200 ms), job success/failure rates, and event processing time

7.3 Performance Targets

- API latency: < 200 ms at p95
- No full table scans on any query path
- Optimised data access patterns throughout

© 2025 Zetheta. For selection purposes only.

8. Deployability

The entire system — all services, databases, and the UI — must be runnable locally via a single command. A clear, complete README is required.

- Local: Docker Compose (mandatory). All services must start with docker compose up.

Cloud or platform deployment is a bonus. Accepted platforms:

- GCP (Google Cloud Platform) or AWS — for full infrastructure deployment
- Vercel or Render — acceptable for frontend/API services; database must be containerised or use a managed service

Provide a live accessible URL in your README if you deploy to any of the above platforms.

---

9. Documentation & Git Commit Requirements

Your Git history is part of your submission. It will be reviewed to understand your engineering process, decision-making, and discipline. The following requirements are mandatory.

9.1 Repository Structure

- All work must be in a single public GitHub repository.
- The repository must include a comprehensive README.md at the root with setup instructions, architecture overview, design decisions, and usage guide.
- A docs/ directory is expected to contain all intermediate planning and design artefacts.

9.2 First Commit — Initial Plan Document

Your very first commit must contain a plan document (docs/plan.md or equivalent). This document must be created before any implementation begins. It should cover:

  - System architecture and component breakdown
  - Technology choices with justification
  - Data flow diagrams or pseudocode sketches
  - Identified risks and open questions
  - Estimated task breakdown and approach

Commits that contain implementation code without a prior plan commit will be penalised in evaluation.

9.3 Intermediate Artefacts

As your implementation progresses, commit intermediary documents that capture your evolving thinking. Expected artefacts include (but are not limited to):

- Schema design document — data models, index strategy, relationship diagram
- API design document — endpoints, request/response contracts, auth flow
- Cross-application token design — mechanism choice, lifecycle, security properties
- Pipeline design — worker logic, retry strategy, idempotency approach
- Deployment design — Docker Compose topology, service dependencies
- Retrospective — a brief learnings note at the end of the project

© 2025 Zetheta. For selection purposes only.

9.4 Commit Hygiene

- Commits must be atomic and descriptive — each commit should represent a single logical unit of work.
- Commit messages must follow a consistent convention (e.g., Conventional Commits: feat:, fix:, docs:, chore:).
- Do not squash all commits into one. The full history will be reviewed.
- Branches are encouraged for feature isolation; merge via pull requests with meaningful descriptions.

---

10. Use of Coding Tools & AI Assistance

You are permitted and encouraged to use coding tools, AI assistants (including large language models), and any libraries or frameworks you deem appropriate. We evaluate engineers on outcomes and judgment, not on avoiding tools.

However, the following conditions are non-negotiable:

- You must be able to explain every part of your submission in a follow-up technical interview. If you cannot explain a piece of code or design, it will be treated as if it does not exist.
- All AI-generated or tool-assisted code must be reviewed, understood, and owned by you. Copy-pasting without comprehension is not acceptable.
- Your documentation and commit messages must reflect your own understanding and reasoning — not the output of a tool.
- Where AI tools were used significantly, briefly note this in your README (e.g., "Used GitHub Copilot for boilerplate scaffolding"). Transparency is valued.

---

11. Security & Implementation Quality

We treat security and code quality as first-class engineering concerns, not afterthoughts. Submissions will be assessed on both.

11.1 Security Expectations

- No secrets, API keys, or credentials committed to the repository under any circumstances. Use environment variables and a .env.example file.
- Input validation and sanitisation on all API endpoints.
- The cross-application token mechanism must be cryptographically sound — no guessable tokens, no client-side generation.
- Basic authentication or access control on the UI/API, even if minimal.
- Dependencies must be pinned and free of known critical vulnerabilities (run a basic audit, e.g., npm audit or pip-audit, and document findings).
- Container images must not run as root.

© 2025 Zetheta. For selection purposes only.

11.2 Code Quality Expectations

- Code must be readable, consistently structured, and well-commented where non-obvious logic exists.
- Tests are expected — unit tests at minimum, integration tests are a strong positive signal. Document how to run them.
- Linting and formatting must be configured (e.g., ESLint, Prettier) and passing.
- Avoid anti-patterns: hardcoded values, deeply nested logic without abstraction, duplicated code without clear reason.

12. Deliverables Summary

Submit your GitHub repository link via the provided submission form. Ensure the repository contains all of the following:

#	Deliverable	Mandatory?
1	Working end-to-end system (all apps + services running via Docker Compose)	**Yes**
2	Candidate Portal with login and Start Assessment trigger	**Yes**
3	Assessment Engine with MCQ interface and submission flow	**Yes**
4	Employer Dashboard with candidate funnel and real-time updates	**Yes**
5	Cross-application token mechanism — server-side, short-lived, single-use	**Yes**
6	Asynchronous evaluation pipeline (PostgreSQL + Redis + worker)	**Yes**
7	Initial plan document in first Git commit (docs/plan.md)	**Yes**
8	Intermediate design documents committed throughout development	**Yes**
9	Root README with setup, architecture overview, and design decisions	**Yes**
10	Test suite with instructions to run	**Yes**
11	Security audit output or notes (e.g., npm audit results)	**Yes**
12	Cloud or platform deployment URL (GCP, AWS, Vercel, or Render)	Bonus

© 2025 Zetheta. For selection purposes only.

13. Evaluation Criteria

Submissions are scored across seven dimensions. Weightings reflect our engineering priorities.

Criterion	Weight	What We Look For
System Design & Architecture	**30%**	Monorepo clarity, service boundaries, schema design, scalability reasoning, documentation quality
Distributed Systems Thinking	**20%**	Pipeline correctness, idempotency, fault tolerance, retry logic, event-driven design
Authentication & Security	**15%**	Cross-app token correctness, security hygiene, input validation, no secrets in repo
Performance & Efficiency	**10%**	API latency, index strategy, no full scans, optimised data access
Fault Tolerance	**10%**	Worker retry, duplicate handling, graceful Redis degradation, partial failure recovery
Code Quality & Structure	**10%**	Readability, test coverage, linting, commit discipline, intermediate artefacts
Observability	**5%**	Structured logging, error tracking, latency and pipeline metrics
Bonus: Cloud Deployment	**+5%**	Live URL on GCP, AWS, Vercel, or Render with accessible, working services

Good luck. We look forward to seeing how you think.

— Zetheta Engineering Team

© 2025 Zetheta. For selection purposes only.